SWINOG #42

The Cloud-Native Advantage: Intent-based Network Automation

Kubernetes already runs our network functions.
It should configure them, too.



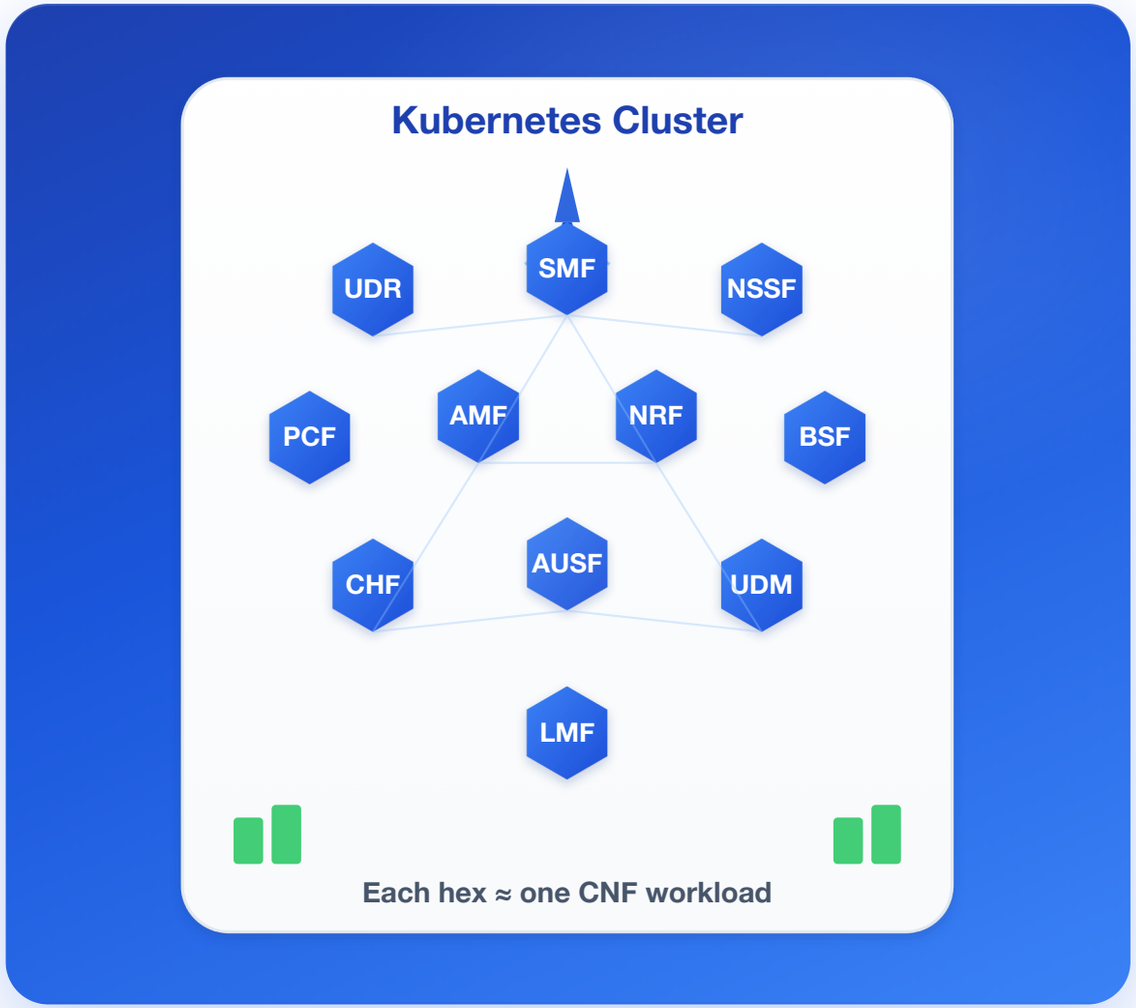
Alexander North
Senior DevOps Engineer · **Swisscom**



Markus Vahlenkamp
Technology & Architecture Consultant · **Nokia**

What is a 5G Mobile Core?

- **Each blue object** is a **CNF** (Cloud-Native Network Function)
  - “Router” (UPF)
  - “Authentication Service” (AUSF)
  - Helm deployed
- **Configuration is done via**
  - Helm Values
  - Configuration interfaces (NETCONF, REST)
- **Scale**
  - Development environment contains **~2000 Pods**
  - **5000 interdependent configuration parameters**



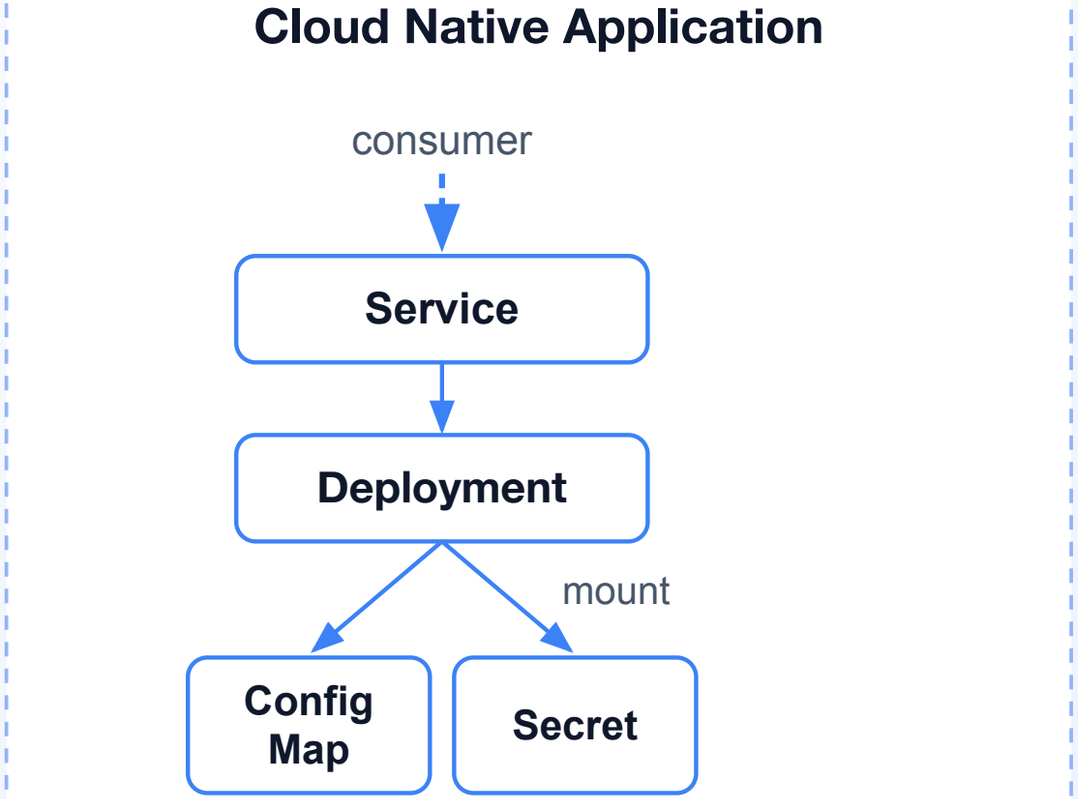
APP | DATABASE | NETWORK FUNCTION | TODAY

A cloud-native app: one declarative step

```
$ kubectl apply -f app.yaml
deployment.apps/app created
```

- Desired state is declared. Kubernetes ensures it is maintained
- Configuration = ConfigMap + Secret, mounted into the pod
- Familiar workflow for every app

✓ fully declarative



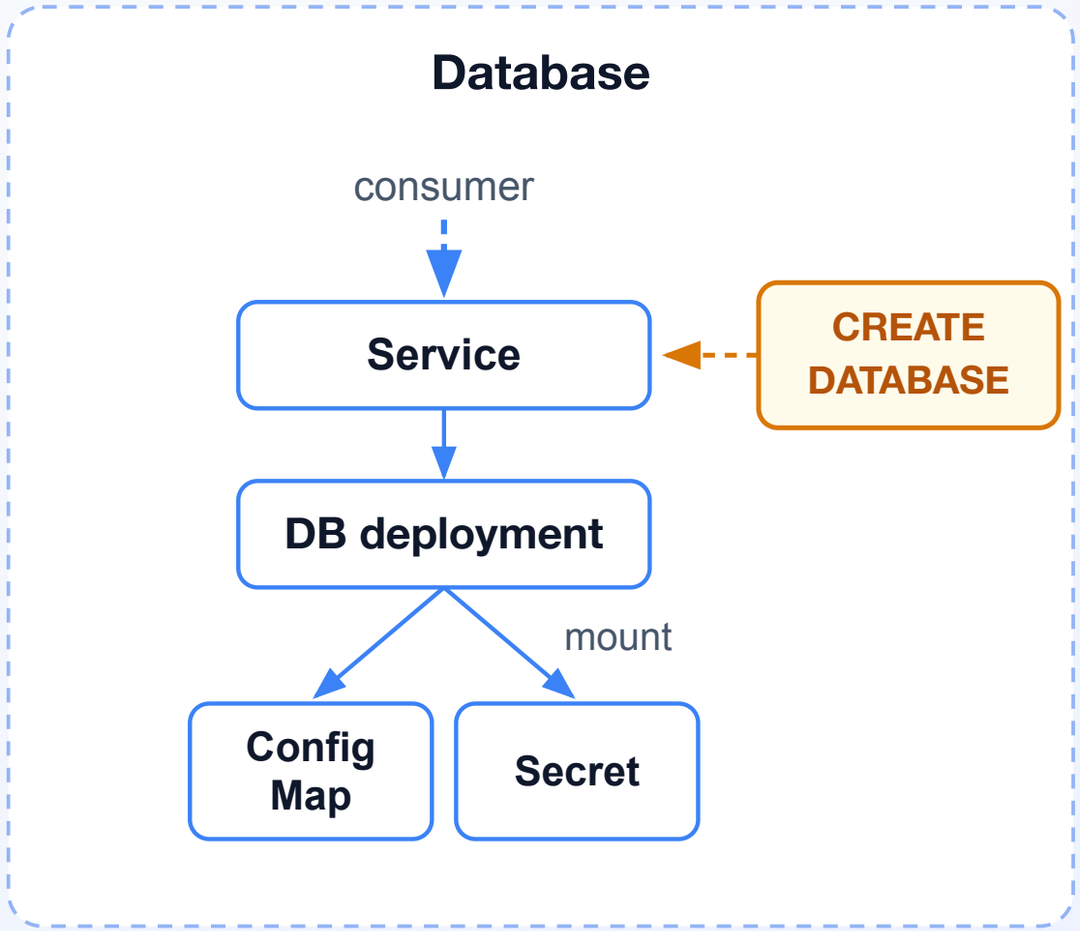
APP | **DATABASE** | NETWORK FUNCTION | TODAY

A database: one extra, imperative step

```
$ kubectl apply -f db.yaml   # the pod
$ psql -c "CREATE DATABASE orders"
```

- The deployment is declarative, but creating the database is a manual day-1 step
- Operators can perform the imperative steps: the database becomes a custom resource

⚠ **partially, operator makes it declarative**



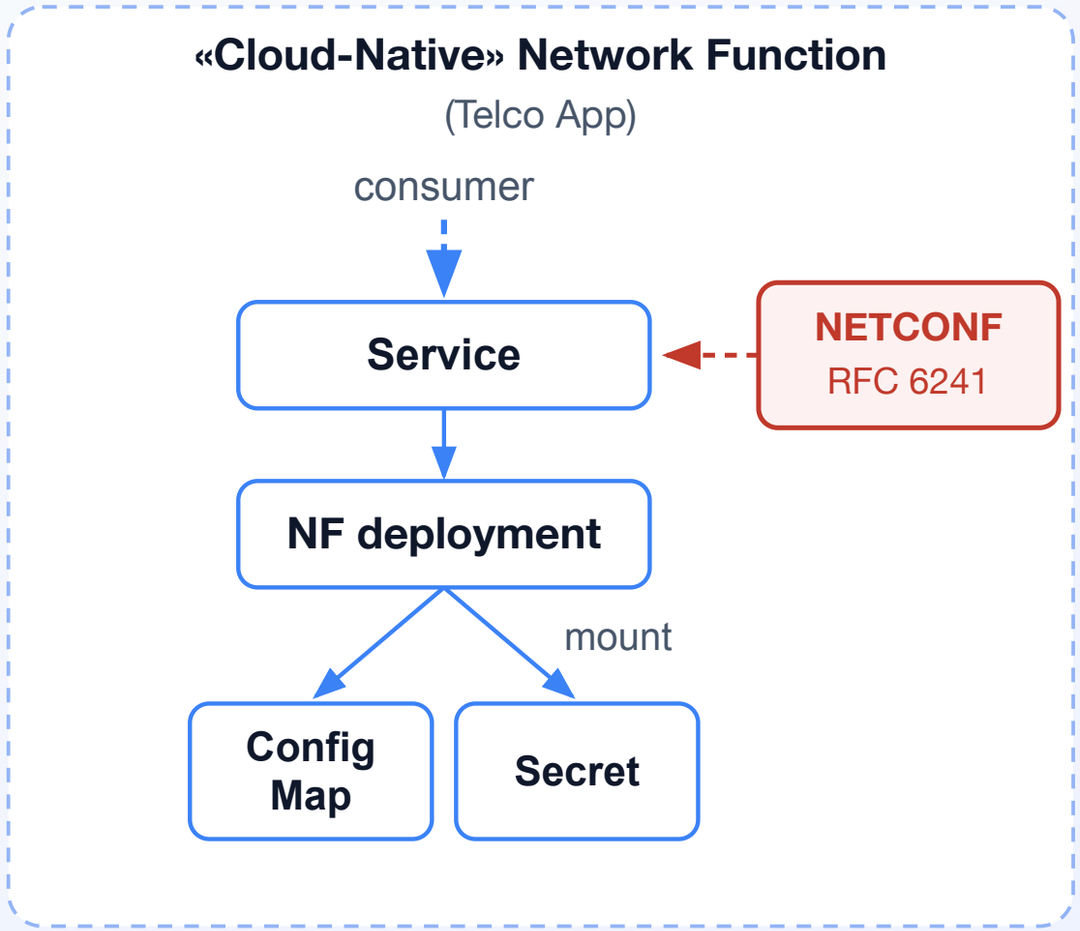
APP | DATABASE | **NETWORK FUNCTION** | TODAY

A network function: a different world

```
$ helm install upf ./upf    # the pod
$ ssh -s upf netconf        # the config
<edit-config>...</edit-config>
```

- Helm deploys the pod, the running config is set through NETCONF and YANG
- Imperative RPCs, needed per instance
- No desired state, no reconcile loop, no familiar cloud-native tooling

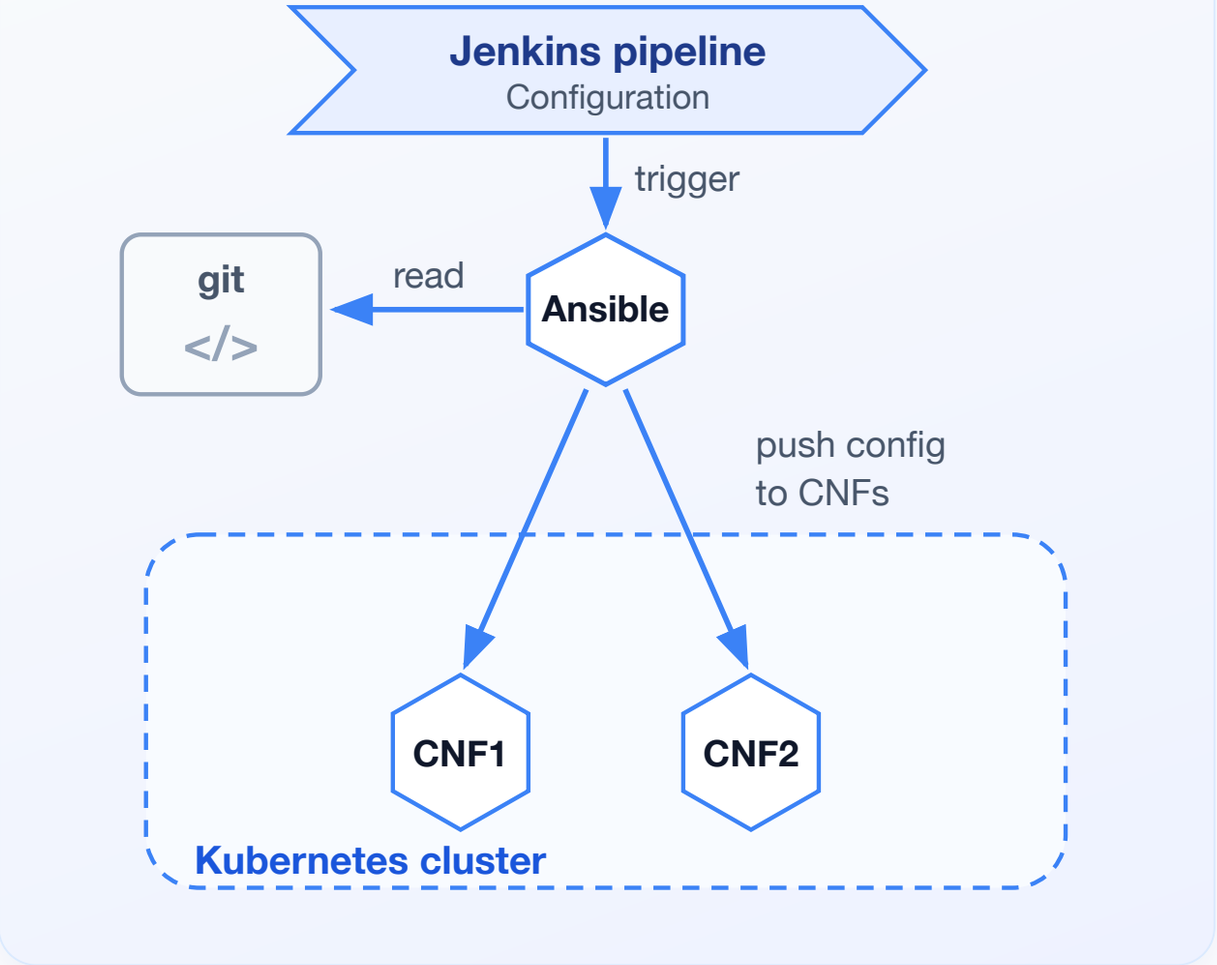
✗ not cloud-native, yet



APP DATABASE NETWORK FUNCTION TODAY

How we configure them today

- **One-shot:** the pipeline pushes once and moves on
- **Slow iterations:** every change goes through the pipeline
- **No monitoring:** changes are not continuously observed
- **Traceability:** decide between multiple sources for auditability and traceability, or one large source for a holistic view



Kubernetes Resource Model (KRM)

API extensions

Custom Resource Definitions extend the Kubernetes API.

```
kind:
CustomResourceDefinition
```

CRs as Instances

Custom Resources instantiate a CRD.

```
kind: Config
name: upf-config
```

Business Logic

Use of Operators or templates to run custom logic.

```
watch → compare → act
```

Same API, same tooling (kubectl, RBAC, GitOps) for anything you can model.

Kubernetes Operators

THE CONTROL LOOP

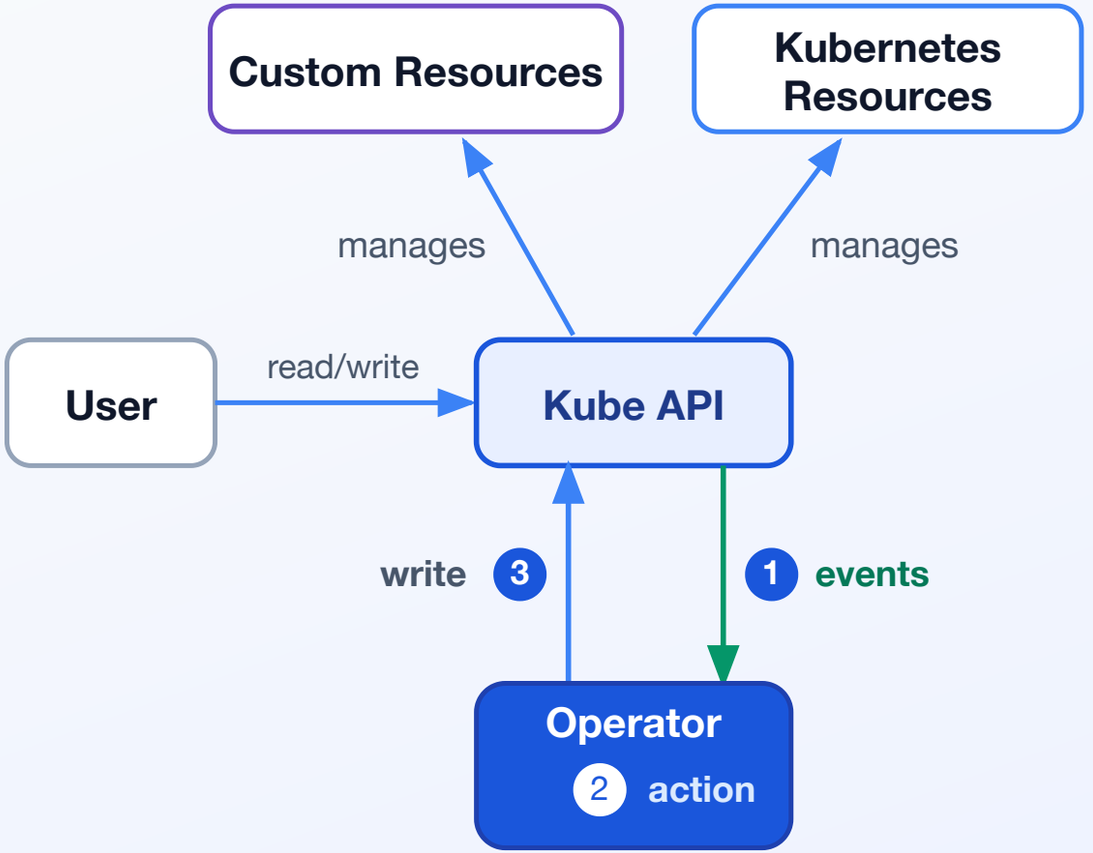
1. Watches (events)
2. Performs action ↻
3. Updates resources (write)

BUILT-IN

Deployment · Job · PersistentVolume

THIRD-PARTY

Prometheus Operator · External Secrets Operator · cert-manager



Network configuration protocols

NETCONF

✓ SUPPORTED BY SDC

TRANSPORT

SSH

ENCODING

XML

gNMI

✓ SUPPORTED BY SDC

TRANSPORT

gRPC

ENCODING

PROTO JSON

JSON_IETF

RESTCONF

NOT SUPPORTED YET

TRANSPORT

HTTP(S)

ENCODING

JSON_IETF XML

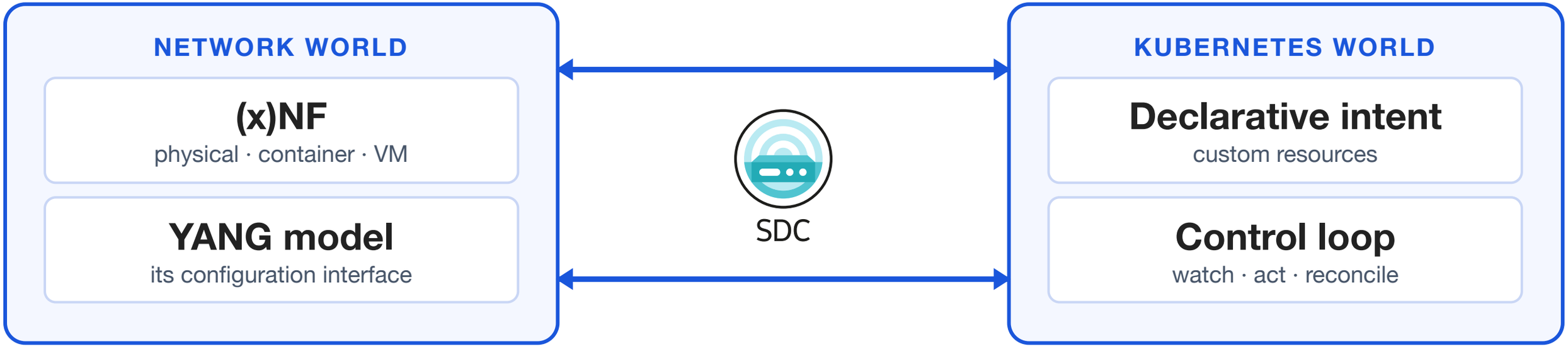
YANG schema

the common base of all three

Conversion between encodings

Validation

Separating the Worlds



Cloud-native network configuration for anything YANG-modelled

- Initiated by Nokia
- Open source since 2024
- CNCF Sandbox since 2026

Configuration as intent

```yaml
kind: Config
metadata:
  name: demo-new-interface
  labels:
    config.sdcio.dev/targetName: demo
spec:
  priority: 90
  revertive: false
  config:
  - path: /
    value:
      network:
        interface:
        - name: eth1
          type: loopback
          ipv4-enabled: true
          ipv4-address: [10.10.10.10]
```

what resource

the `Config` CR kind

which intent

this object's `metadata.name`

which device

label pointing at the Target

who wins

if two Configs set the same leaf

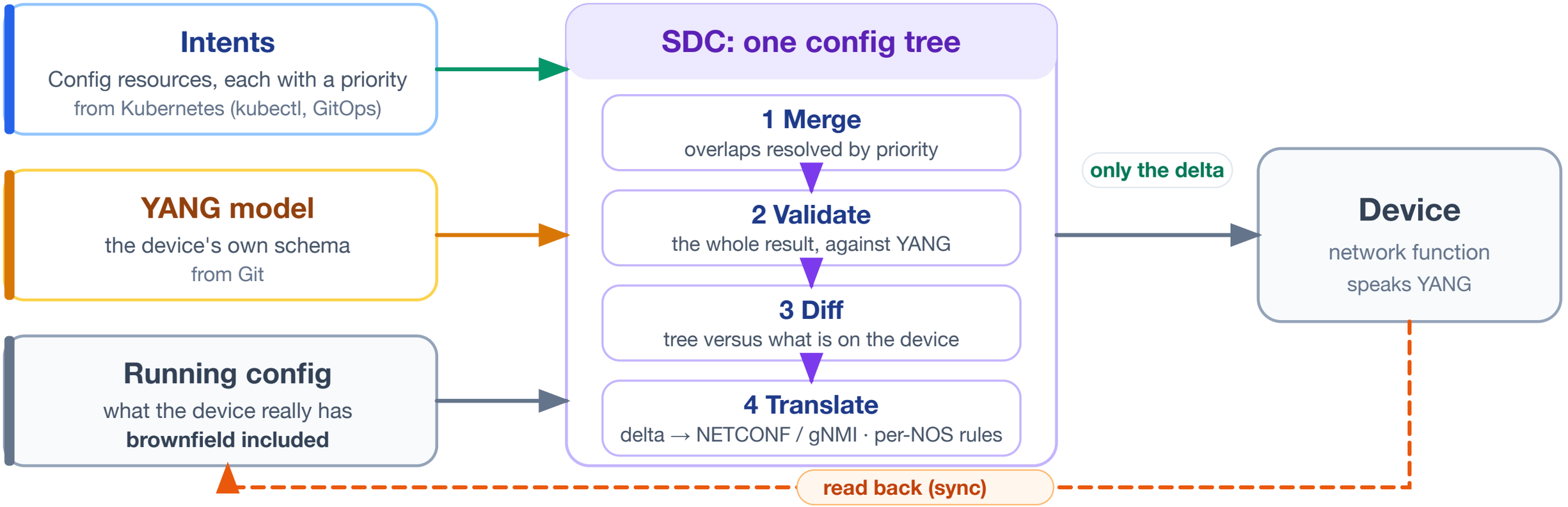
what if it drifts

`false` — drift is reported, not auto-healed

what to set

only your part of the YANG tree

An intent's journey to the device



Intent resolution

initial-config Platform · baseline	/network/interface[name=eth0]/ipv4-address /network/interface[name=eth0]/speed	**192.0.2.10** **10G**	prio 100
~~overlap-config~~ ~~Ops · temp override~~	~~/network/interface[name=eth0]/ipv4-address~~ ~~/network/interface[name=eth0]/admin-up~~	~~20.20.20.20~~ ~~true~~	~~prio~~ ~~50~~

DEVICE (MERGED RESULT)

/network/interface[name=eth0]/ipv4-address	**20.20.20.20**	prio 50 wins
/network/interface[name=eth0]/speed	**10G**	prio 100
/network/interface[name=eth0]/admin-up	**true**	prio 50

The **lower** the priority number, the **higher** the precedence.

Without competition, the result is simply a **merge** of all intents.

Delete `overlap-config`: back to `192.0.2.10`, nothing was lost.

When the device drifts

```
kind: Deviation
metadata:
  name: demo-initial-config
spec:
  deviations:
  - path: network/service[name=svc-b]/bandwidth
    desiredValue: 5000
    actualValue: 100
    reason: NOT_APPLIED
```

Reason	Meaning	SDC does
`NOT_APPLIED`	intended value is not on the device	**reports only** revertive: re-applies
`OVERRULED`	a higher-priority Config wins	**reports only**
`UNHANDLED`	brownfield config, no Config owns it	**reports only**

Where does this value come from?

running + **intended** + **deviation** in one tree, each value tagged with its **owner**

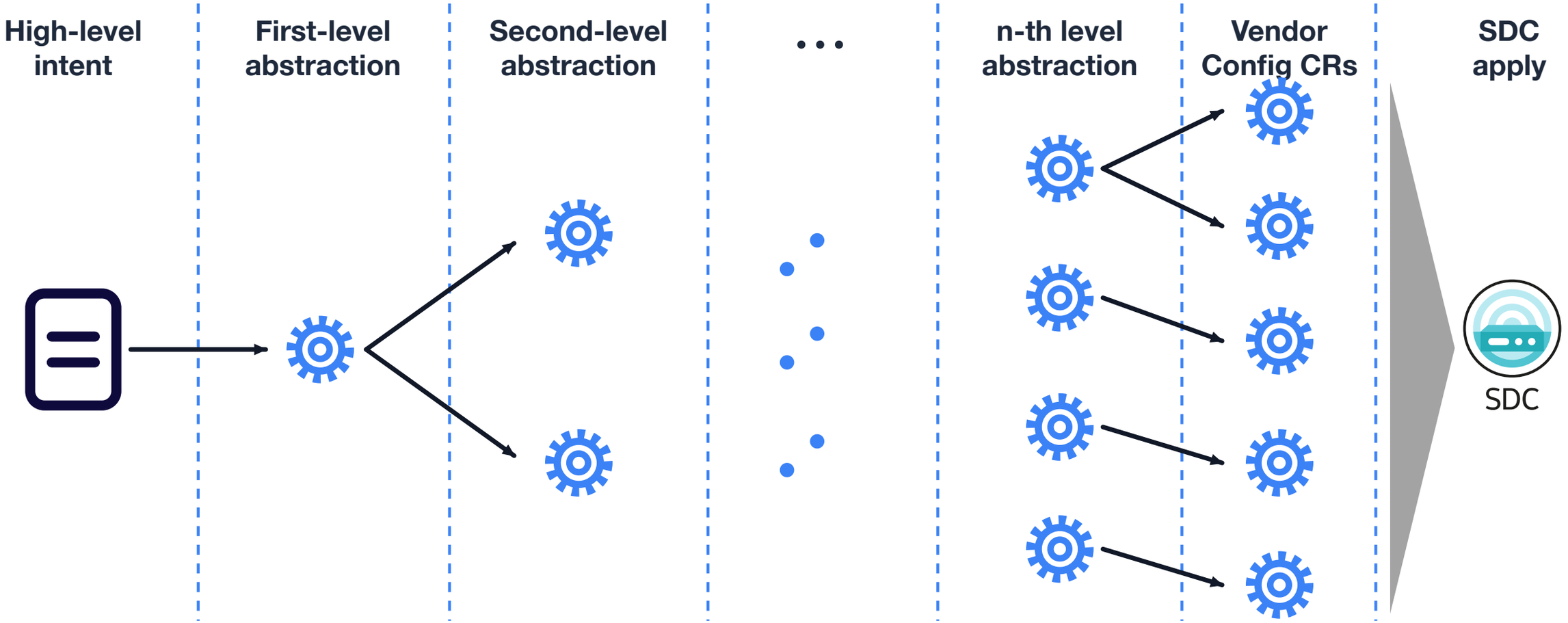
Config Blame is to Config apply what Git Blame is to commit.

```
$ kubectl sdc blame --target demo
                        -----  🎯 demo.demo
                        -----  └── 📦 network
                        -----      ├── 📦 interface
                        -----      │   ├── 🔑 name=eth0
          demo.initial-config      │   │   ├── 🌿 speed -> 10G
          demo.overlap-config      │   │   ├── 🌿 admin-up -> true
          demo.overlap-config      │   │   └── 🌿 ipv4-address -> 20.20.20.20
                        -----      │   └── 🔑 name=eth1
     demo.demo-new-interface       │   │   ├── 🌿 ipv4-enabled -> true
     demo.demo-new-interface       │   │   └── 🌿 ipv4-address -> 10.10.10.10
                        -----      │   └── 📦 service
                        -----      │       ├── 🔑 name=svc-b
      demo.initial-config(*)       │       │   └── 🌿 bandwidth -> 100 [~> 5000]
                        -----      │       └── 🔑 name=svc-c
                      running      │           └── 🌿 bandwidth -> 500
```

Left: **owner** (`namespace.config` or `running` = brownfield) · `-> 100` actual · `[~> 5000]` expected · `(*)` deviation

Demo

From high-level intent to device config



What this gets you

- **Less toil**: describe intent, not commands
- **Reliability**: drift detected and corrected
- **Visibility**: running state, deviations, source
- **Scale**: composition by priority and overlapping sets, not copy/paste

Try it

docs.sdcio.dev

- Quick start & lab
- GitHub: github.com/sdcio
- Discord for news and questions
- Community meetup, Mondays 08:30 CET

Alexander North · alexander.north1@swisscom.com
Markus Vahlenkamp · markus.vahlenkamp@nokia.com

